

Loss-Based Coloring of Neural Networks

Mathematical Details

MNIST Symmetries Project

Contents

1	Overview	1
2	Setup: Network and Notation	2
3	Kronecker-Factored Curvature Matrices	2
4	The Clustering Objective	3
5	Optimal Representative	4
5.1	Global Optimum (Coupled System)	4
5.2	Local (Within-Cluster) Approximation	4
6	Merge Delta	4
7	Multi-Layer Shared Budget	5
8	Greedy Algorithm	5
9	Sparsification of S	6
10	Collapsing the Network	6
10.1	Representative Weight: Row-Sum Formula	7
10.2	First Hidden Layer ($\ell = 1$)	7
10.3	Subsequent Hidden Layers ($\ell > 1$)	7
10.4	Output Layer	8
11	Summary of Quantities per Layer	8

1 Overview

Loss coloring is a method for partitioning the hidden nodes of a trained MLP into groups (colors, or clusters) such that all nodes in the same group can be replaced by a single representative node with the smallest possible increase in the training loss.

Unlike coloring based on fibration symmetries—which uses weight-matrix geometry—loss coloring uses the *curvature of the loss landscape* to measure the cost of merging nodes. The

curvature information is captured by two Kronecker-factored matrices, A and S , borrowed from the EigenDamage framework [1].

2 Setup: Network and Notation

Consider an MLP with L hidden layers. Let layer ℓ have n_ℓ^{in} inputs and n_ℓ^{out} outputs. Denote:

- $a^{(\ell)} \in \mathbb{R}^{N \times n_\ell^{\text{in}}}$: matrix of pre-ReLU activations entering layer ℓ over a mini-batch of N samples.
- $z^{(\ell)} \in \mathbb{R}^{N \times n_\ell^{\text{out}}}$: matrix of pre-activation outputs of layer ℓ (i.e. $z^{(\ell)} = a^{(\ell)}W^{(\ell)\top} + \mathbf{1}b^{(\ell)\top}$).
- $g_s^{(\ell)} = \partial\mathcal{L}/\partial z^{(\ell)} \in \mathbb{R}^{N \times n_\ell^{\text{out}}}$: gradient of the loss with respect to the pre-activations.
- $W^{(\ell)} \in \mathbb{R}^{n_\ell^{\text{out}} \times n_\ell^{\text{in}}}$: weight matrix.
- $b^{(\ell)} \in \mathbb{R}^{n_\ell^{\text{out}}}$: bias vector.

To treat weight and bias jointly, define the *augmented* weight vector for output node j of layer ℓ :

$$w_j^{(\ell)} = [W_{j,:}^{(\ell)} \mid b_j^{(\ell)}] \in \mathbb{R}^{n_\ell^{\text{in}}+1},$$

assembled into the augmented weight matrix $w^{(\ell)} \in \mathbb{R}^{n_\ell^{\text{out}} \times (n_\ell^{\text{in}}+1)}$.

Correspondingly, the augmented activation is

$$a_{\text{aug}}^{(\ell)} = [a^{(\ell)} \mid \mathbf{1}_N] \in \mathbb{R}^{N \times (n_\ell^{\text{in}}+1)},$$

where $\mathbf{1}_N$ is an N -dimensional column of ones (accounting for the bias).

3 Kronecker-Factored Curvature Matrices

For each hidden layer ℓ , two symmetric positive semi-definite matrices are computed from one forward-backward pass on a mini-batch:

$$A^{(\ell)} = \frac{1}{N} a_{\text{aug}}^{(\ell)\top} a_{\text{aug}}^{(\ell)} \in \mathbb{R}^{(n_\ell^{\text{in}}+1) \times (n_\ell^{\text{in}}+1)}, \quad (1)$$

$$S^{(\ell)} = \frac{1}{N} g_s^{(\ell)\top} g_s^{(\ell)} \in \mathbb{R}^{n_\ell^{\text{out}} \times n_\ell^{\text{out}}}. \quad (2)$$

Interpretation. $A_{kk'}^{(\ell)} = \mathbb{E}[a_{\text{aug},k}^{(\ell)} a_{\text{aug},k'}^{(\ell)}]$ is the second-order statistics of the incoming activations. $S_{jj'}^{(\ell)} = \mathbb{E}[g_{s,j}^{(\ell)} g_{s,j'}^{(\ell)}]$ measures the co-sensitivity of the loss to the pre-activations of nodes j and j' . In particular, $S_{jj}^{(\ell)}$ is large when the loss is highly sensitive to node j , and $S_{jj'}^{(\ell)}$ is large when nodes j and j' are simultaneously sensitive (their gradients co-vary).

Connection to the Fisher information. Under the Kronecker-factored approximation of the Fisher information matrix $\mathcal{F} \approx A \otimes S$ (EigenDamage, [1]), the second-order change in loss when the j -th row of $w^{(\ell)}$ is shifted by δw_j is approximately

$$\Delta \mathcal{L}_j \approx \frac{1}{2} S_{jj}^{(\ell)} \delta w_j^\top A^{(\ell)} \delta w_j.$$

When merging nodes j and j' into a single representative r_c , the shifts are $\delta w_j = r_c - w_j$ for every j in cluster c .

4 The Clustering Objective

Let $\mathcal{C} = \{c_1, c_2, \dots\}$ be a partition of the output nodes of layer ℓ (we drop the layer superscript for clarity). Let $r_c \in \mathbb{R}^{n_\ell^{\text{in}}+1}$ be the representative of cluster c . Define the total cost:

$$F(\mathcal{C}, \{r_c\}) = \sum_{j, j'=1}^n S_{jj'} (w_j - r_{c(j)})^\top A (w_{j'} - r_{c(j')}) \quad (3)$$

where $c(j)$ denotes the cluster containing node j , and $n = n_\ell^{\text{out}}$.

Remark 1. *This is the full bilinear form weighted by S . When S is diagonal, the cross terms $S_{jj'}$ for $j \neq j'$ vanish and one recovers the simpler sum of diagonal EigenDamage costs $\sum_j S_{jj} (w_j - r_{c(j)})^\top A (w_j - r_{c(j)})$.*

Expanding (3) by grouping same-cluster and cross-cluster terms:

$$F = \underbrace{\sum_c \sum_{j, j' \in c} S_{jj'} (w_j - r_c)^\top A (w_{j'} - r_c)}_{F_{\text{within}}} + \underbrace{\sum_{c \neq c'} \sum_{j \in c, j' \in c'} S_{jj'} (w_j - r_c)^\top A (w_{j'} - r_{c'})}_{F_{\text{cross}}}. \quad (4)$$

Expanding and collecting by cluster:

$$F = \sum_c \left[M_c r_c^\top A r_c - 2 r_c^\top A \beta_c \right] + 2 \sum_{c < c'} M_{cc'} r_c^\top A r_{c'} + \text{const}, \quad (5)$$

where the constant does not depend on the coloring or representatives, and:

$$M_c = \sum_{j, j' \in c} S_{jj'}, \quad (6)$$

$$M_{cc'} = \sum_{j \in c, j' \in c'} S_{jj'}, \quad (7)$$

$$\beta_c = \sum_{j \in c} \sum_{j'=1}^n S_{jj'} w_{j'} = (S w)_{c,:} \quad (\text{rows of } S w \text{ summed over cluster } c). \quad (8)$$

The per-cluster diagonal term in the code is:

$$\text{F_diag}(M_c, r_c, \beta_c, A) = M_c r_c^\top A r_c - 2 r_c^\top A \beta_c.$$

In the code, β_c is accumulated as `b_global[c] =  $\sum_{j \in c} (S w)_j$` , and $M_{cc'}$ is stored in `M_cross[c][c']`.

5 Optimal Representative

5.1 Global Optimum (Coupled System)

Taking $\partial F / \partial r_c = 0$ from (5):

$$M_c A r_c + \sum_{c' \neq c} M_{cc'} A r_{c'} = A \beta_c,$$

which couples all cluster representatives. Solving this global system is expensive; instead the code uses a local approximation.

5.2 Local (Within-Cluster) Approximation

The representative is optimized considering only the within-cluster terms of F , i.e. treating the cross-cluster contributions as fixed:

$$r_c^* = \frac{\beta_c^{\text{loc}}}{M_c},$$

where:

$$\beta_c^{\text{loc}} = \sum_{j \in c} \sum_{j' \in c} S_{jj'} w_{j'}, \quad (9)$$

$$M_c = \sum_{j, j' \in c} S_{jj'}. \quad (10)$$

That is, r_c^* is the weighted average of the weight vectors of nodes in c , where the weight of node j' is its total within-cluster connectivity $\sum_{j \in c} S_{jj'}$.

In the code:

- `b_local[c]` accumulates β_c^{loc} .
- `M_local[c]` accumulates M_c .
- `rep[c] = b_local[c] / M_local[c]`.

6 Merge Delta

The core of the greedy algorithm is computing the change in F when two clusters a and b are merged into a new cluster $c = a \cup b$.

Only the terms in (5) that involve a or b change. Let $\mathcal{N}$ be the set of clusters neighboring a or b (but not a or b themselves).

Cost before merge.

$$\begin{aligned} F_{\text{before}} = & \text{F_diag}(M_a, r_a, \beta_a, A) + \text{F_diag}(M_b, r_b, \beta_b, A) + 2 M_{ab} r_a^\top A r_b \\ & + 2 \sum_{k \in \mathcal{N}} (M_{ak} r_a^\top A r_k + M_{bk} r_b^\top A r_k). \end{aligned} \quad (11)$$

Merged cluster state.

$$M_c = M_a + M_b + 2M_{ab}, \quad (12)$$

$$\beta_c^{\text{loc}} = \beta_a^{\text{loc}} + \beta_b^{\text{loc}} + b_{ab} + b_{ba}, \quad (13)$$

$$\beta_c = \beta_a + \beta_b \quad (\text{b_global accumulates linearly}), \quad (14)$$

$$r_c^* = \beta_c^{\text{loc}} / M_c, \quad (15)$$

$$M_{ck} = M_{ak} + M_{bk} \quad \forall k \in \mathcal{N}, \quad (16)$$

where $b_{ab} = \sum_{j \in a, j' \in b} S_{jj'} w_{j'} = \text{b_cross}[a][b]$ and b_{ba} symmetrically.

Cost after merge.

$$F_{\text{after}} = \text{F_diag}(M_c, r_c^*, \beta_c, A) + 2 \sum_{k \in \mathcal{N}} M_{ck} r_c^{*\top} A r_k. \quad (17)$$

Merge delta.

$$\boxed{\delta(a, b) = F_{\text{after}} - F_{\text{before}}} \quad (18)$$

A negative δ means merging a and b *decreases* the loss approximation error — these merges are beneficial and should be prioritized.

7 Multi-Layer Shared Budget

For a network with L hidden layers, the total cost is the sum over all layers:

$$F_{\text{total}} = \sum_{\ell=1}^L F^{(\ell)}.$$

A single budget F_{max} is shared across all layers. The algorithm finds the coloring $\mathcal{C}^{(1)}, \dots, \mathcal{C}^{(L)}$ such that:

$$F_{\text{total}} \leq F_{\text{max}},$$

while maximizing the number of merged nodes (minimizing the total number of colors).

8 Greedy Algorithm

Lazy deletion. When a node is merged, old heap entries involving it become stale. Instead of removing them (expensive), the algorithm checks at pop time whether both endpoints are still active; stale entries are discarded.

Recompute before commit. Because δ is recomputed with the current state (after other merges may have changed neighbor representatives), the committed δ is always accurate. This makes the budget check exact.

Algorithm 1 Multi-layer loss coloring with shared budget

```
1: Initialize one LayerState per layer  $\ell$  (builds  $M_{\text{local}}, \beta^{\text{loc}}, \beta, r, M_{\text{cross}}, \beta_{\text{cross}}$  from  $w^{(\ell)}$  and  $S^{(\ell)}$ ).
2: Push all candidate pairs  $(a, b)$  from all layers onto a min-heap  $\mathcal{H}$ , keyed by  $\delta(a, b)$ .
3:  $F_{\text{total}} \leftarrow 0$ .
4: while  $\mathcal{H}$  is not empty do
5:   Pop  $(\hat{\delta}, \ell, a, b) \leftarrow \min \mathcal{H}$ .
6:   if  $a$  or  $b$  is no longer active (stale entry) then continue
7:   end if
8:   Recompute  $\delta(a, b)$  with current state (stale  $\hat{\delta}$  may differ).
9:   if  $F_{\text{total}} + \delta > F_{\text{max}}$  then break
10:  end if
11:  Merge  $a$  and  $b$  into new node  $c$ ; update state (eqs. (12)–(16)).
12:   $F_{\text{total}} \leftarrow F_{\text{total}} + \delta$ .
13:  Push new candidates  $(c, k)$  for all  $k \in \mathcal{N}$  onto  $\mathcal{H}$ .
14: end while
15: return clusters and representatives for each layer.
```

Termination. The loop breaks as soon as the cheapest available merge (across all layers) would exceed the shared budget. Because the heap is a global min-heap, this is the globally cheapest remaining merge, so the stopping condition is tight.

9 Sparsification of S

Since $S^{(\ell)} = (g_s^{(\ell)})^\top g_s^{(\ell)} / N$ is a dense $n_\ell^{\text{out}} \times n_\ell^{\text{out}}$ matrix, naively all $\binom{n_\ell^{\text{out}}}{2}$ node pairs are candidate merges, making heap initialization $O(n^2)$ in cost.

To reduce complexity, entries below the p -th percentile of $|S^{(\ell)}|$ are zeroed before building the initial structures:

$$S_{jj'}^{(\ell)} \leftarrow 0 \quad \text{if} \quad |S_{jj'}^{(\ell)}| < \text{percentile}_p(|S^{(\ell)}|).$$

Justification. If $S_{jj'} \approx 0$, the gradients of nodes j and j' are nearly uncorrelated. These nodes do not interact strongly in the objective F , so dropping their cross-term introduces negligible error while making the graph sparse. With $p = 90$ (default), roughly the top 10% of entries are retained per layer, reducing heap size from $O(n^2)$ to $O(n \cdot k)$ where k is the average number of retained neighbors.

10 Collapsing the Network

Once the coloring $\{c(j)\}$ is determined, the network is collapsed: each cluster c is replaced by a single output node with representative weight vector r_c .

10.1 Representative Weight: Row-Sum Formula

For cluster c of layer ℓ , let $c_{\text{idx}} \subseteq \{1, \dots, n\}$ be the indices of its member nodes. Define the within-cluster row-sum weights:

$$\rho_j = \sum_{j' \in c_{\text{idx}}} S_{jj'}^{(\ell)}, \quad j \in c_{\text{idx}}, \quad M_c = \sum_{j \in c_{\text{idx}}} \rho_j.$$

The collapsed weight and bias for cluster c are:

$$r_c^W = \frac{1}{M_c} \sum_{j \in c_{\text{idx}}} \rho_j W_{j,:}^{(\ell)} = \frac{\boldsymbol{\rho}^\top W_{c_{\text{idx}},:}^{(\ell)}}{M_c}, \quad (19)$$

$$r_c^b = \frac{1}{M_c} \sum_{j \in c_{\text{idx}}} \rho_j b_j^{(\ell)} = \frac{\boldsymbol{\rho}^\top b_{c_{\text{idx}}}^{(\ell)}}{M_c}. \quad (20)$$

This is exactly the local representative $r_c^* = \beta_c^{\text{loc}}/M_c$ from (15), written in terms of the original weight matrices.

Remark 2. When $M_c < \varepsilon$ (all S entries in the subblock are zero or near zero, e.g. because of sparsification), the uniform fallback $\rho_j = 1 \forall j$, $M_c = |c|$ is used, which gives the simple arithmetic mean.

10.2 First Hidden Layer ($\ell = 1$)

The input dimension does not change, so:

$$\widetilde{W}_{c,:}^{(1)} = r_c^W \in \mathbb{R}^{n_0}, \quad \widetilde{b}_c^{(1)} = r_c^b.$$

10.3 Subsequent Hidden Layers ($\ell > 1$)

The input to layer ℓ has been collapsed by the coloring of layer $\ell - 1$ to $K_{\ell-1}$ dimensions, where cluster c' of layer $\ell - 1$ acts as a single input unit. The weight from input cluster c' to output cluster c is:

$$\widetilde{W}_{c,c'}^{(\ell)} = \sum_{\substack{i \in c_{\text{idx}}^{(\ell)} \\ j: c^{(\ell-1)}(j) = c'}} \frac{\rho_i}{M_c} W_{i,j}^{(\ell)}, \quad (21)$$

i.e. the c' -th component of the collapsed weight vector r_c^W is the sum of all components of r_c^W that belonged to cluster c' in the previous layer. In code this is:

$$w_{\text{avg}} = \boldsymbol{\rho}^\top W_{c_{\text{idx}},:}^{(\ell)} / M_c \in \mathbb{R}^{n_{\ell-1}^{\text{out}}},$$

$$\widetilde{W}_{c,c'}^{(\ell)} += w_{\text{avg}}[i] \quad \text{for each } i \text{ with } c^{(\ell-1)}(i) = c'.$$

10.4 Output Layer

The output layer is not clustered (its nodes are class logits). Only its *input dimension* changes, since the last hidden layer has been collapsed. For input cluster c of the last hidden layer:

$$\widetilde{W}_{:,c}^{(L+1)} = \frac{W_{:,c_{\text{idx}}}^{(L+1)} \boldsymbol{\rho}}{M_c} \in \mathbb{R}^{n_{\text{out}}}.$$

The output bias is unchanged.

11 Summary of Quantities per Layer

Symbol	Shape	Meaning
$A^{(\ell)}$	$(n_\ell^{\text{in}} + 1) \times (n_\ell^{\text{in}} + 1)$	Input activation covariance (augmented)
$S^{(\ell)}$	$n_\ell^{\text{out}} \times n_\ell^{\text{out}}$	Output gradient covariance
$w^{(\ell)}$	$n_\ell^{\text{out}} \times (n_\ell^{\text{in}} + 1)$	Augmented weight matrix $[W b]$
M_c	scalar	Total within-cluster S mass
$M_{cc'}$	scalar	Cross-cluster S mass
β_c^{loc}	$n_\ell^{\text{in}} + 1$	Within-cluster weighted sum of rows of w
β_c	$n_\ell^{\text{in}} + 1$	All-neighbors weighted sum $(Sw)_{c,:}$
r_c^*	$n_\ell^{\text{in}} + 1$	Optimal representative $= \beta_c^{\text{loc}}/M_c$
$\delta(a, b)$	scalar	Change in F from merging clusters a and b

References

- [1] T. George, C. Laurent, X. Bouthillier, N. Ballas, P. Vincent, *Fast Approximate Natural Gradient Descent in a Kronecker-Factored Eigenbasis*, NeurIPS 2018.